

Tarea 2: Simulador Gráfico de ELOTelTags y Aplicación Find My

Lea detenidamente la tarea. **Si algo no lo entiende, consulte al profesor.** Esta interacción se asemeja a la interacción entre desarrolladores y clientes con el fin de aclarar requerimientos de una aplicación.

1. Objetivos de la tarea

- Modelar objetos reales como objetos de software.
- Ejercitar la creación y extensión de clases dadas para satisfacer nuevos requerimientos.
- Reconocer clases y relaciones entre ellas en códigos fuentes Java.
- Ejercitar la configuración de un ambiente de trabajo para desarrollar aplicaciones JavaFX. Se debe trabajar con un IDE (Integrated Development Environment). Si bien IntelliJ es el sugerido, su grupo puede trabajar con otro IDE.
- Manejar proyectos de software vía GIT.
- Ejercitar la preparación y entrega de resultados de software (readme, documentación).
- Familiarización con desarrollos "iterativos" e "incrementales".

2. Descripción General

Esta tarea busca adaptar y extender la Tarea 1 para simular gráficamente, usando JavaFX, la interacción entre dispositivos de rastreo EloTelTags, celulares y tablets presentados en la Tarea 1.

La descripción del funcionamiento de EloTelTags, celulares y tablets es la misma de la Tarea 1, revise allí esos detalles. Por simplicidad, use pixeles como unidad para las coordenadas de posición y pixeles por segundo para la rapidez de movimiento de los equipos.

3. Funcionamiento de la Aplicación

En esta oportunidad se espera que su programa se ejecute desde un IDE (IntelliJ de preferencia o alternativamente Visual Studio Code).

Una gran diferencia con la Tarea 1 la genera el nuevo paradigma usado en Tarea 2: “programación conducida por eventos”. En la Tarea 1, el método **main** activaba todos los cambios del programa. En esta tarea los eventos del usuario y los del programa gobernarán la ejecución del programa.

Como en la Tarea 1, la configuración de los dispositivos es tomada de un archivo (por ejemplo config.txt) el cual es especificado a través de una instancia de FileChooser, ver código de ayuda de la primera etapa. El formato para ese archivo es:

```
<archivo_imagen_de_fondo>
<delta_tiempo>
<número de personas>
<nombre 1º persona> <número de EloTelTags> (0 | 1)
<celular_x> <celular_y> <rapidez> <ángulo> <delta_ángulo>
(<nombre EloTelTag> <tag_x> <tag_y> <rapidez> <ángulo> <delta_ángulo>) *
[<tablet_x> <tablet_y> <rapidez> <ángulo> <delta_ángulo>]
```

La primera línea contiene la imagen a utilizar como fondo para el territorio donde se mueven los equipos. Para su conveniencia, puede tomar [ésta](#).

delta_tiempo corresponde al valor discreto en segundos en que avanza el tiempo de simulación, por ejemplo, para mover los equipos (tags, celulares, tablets).

Para cada equipo, las coordenadas corresponden a su posición inicial (en unidad de pixeles), la rapidez es la magnitud de la velocidad (en pixeles/s) con que se mueve por el territorio, el ángulo (en grados) es el inicial de su velocidad, y delta_ángulo (en grados) define el rango (-delta_ángulo, +delta_ángulo) de donde aleatoriamente y a intervalos regulares delta_tiempo se modifica el ángulo de la velocidad del equipo y el equipo se desplaza levemente.

Un ejemplo para este archivo es:

Placeres.jpg

0.1

3

Pedro 2 1

1000 100 20 30 8

maleta 50 170 15 -20 10 mochila 120 130 10 -130 30

305 99 15 150 20

Juan 1 0

500 260 15 -120 20

llaves 251 258 10 -90 10

Diego 2 1

335 355 10 -20 20

llaves 400 100 40 90 30 mochila 405 431 15 -10 20

210 310 15 180 20

Al inicio del programa, se muestra una interfaz como la Figura 1.

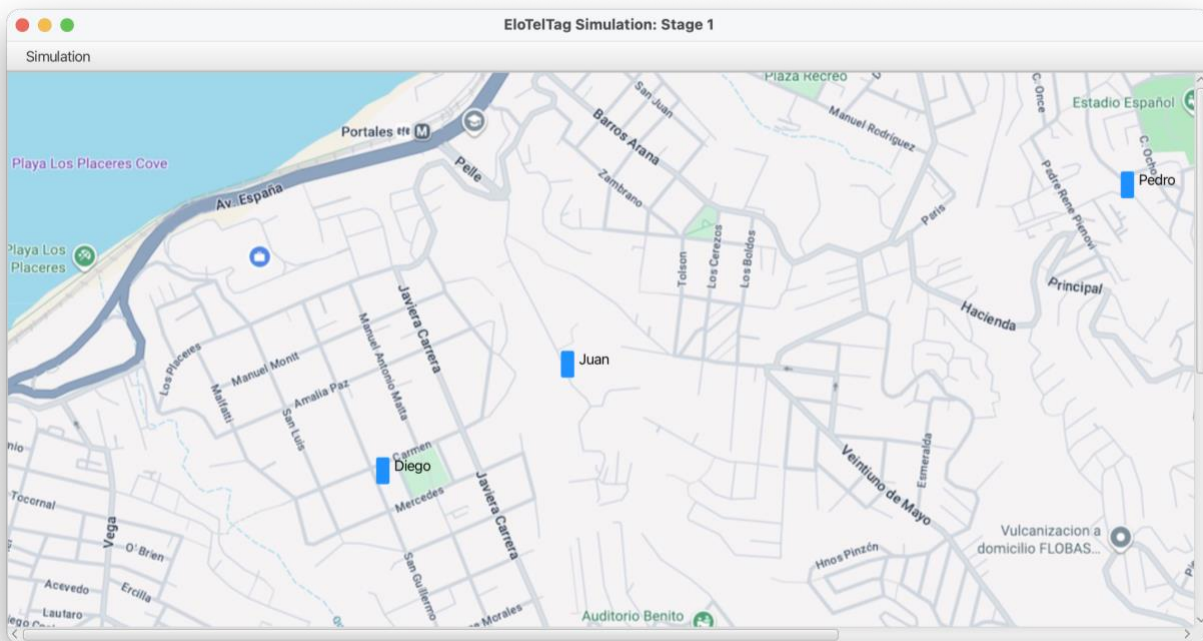


Figura 1: Ejemplo de interfaz con sólo tres celulares

El menú Simulation ofrece dos opciones: “Play” y “Pause” al estilo de la Figura 2.



Figura 2: Opciones del menú Simulation

Al seleccionar “Play” los equipos se mueven por el territorio manteniéndose confinados a él. Al llegar al borde del Territorio, el cual coincide con el tamaño de la imagen usada, los equipos rebotan en el borde. Al rebotar mantienen la componente de la velocidad en sentido paralelo al borde e invierten la componente de la velocidad perpendicular al borde.

Durante la simulación, los EloTelTags y las tablets buscan celulares cercanos cada t_{bip} segundos sin mantener una sincronización de fase (no lo hacen al mismo tiempo). Este tiempo es de 4 segundos para todos los Tags y de 5 segundos para todos los tablets. En la interfaz, la búsqueda de celulares se manifiesta con la emisión de un [sonido corto](#) y un círculo que crece desde el equipo hasta llegar a un

radio de 50 píxeles en 1 segundo. Cuando un celular es alcanzado por la señal de un tag o tablet, como la Tarea 1, éste reporta a la nube el nombre del dispositivo, su dueño/a y la posición del celular en ese momento.

La función FindMy de tablets y celulares se activa presionando en el interior de la vista del equipo. Se muestra así un menú emergente/flotante con una única opción como se muestra en Figura 3. Al seleccionar la opción “Find My” se muestra una nueva ventana con la misma información descrita para la Tarea 1.

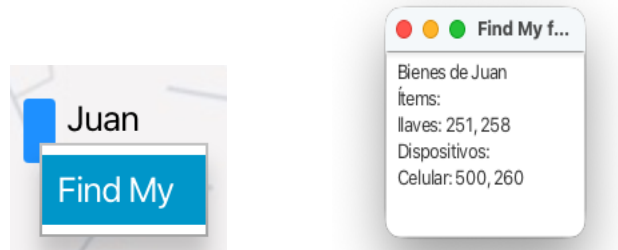


Figura 3: Izquierda menú emergente al presionar en celular o tablet y a la derecha ventana flotante con la información Find My de esa persona

Mientras se mantiene activa la ventana FindMy, ésta se actualiza cada un segundo con la información que corresponda según el movimiento de los equipos.

4. Desarrollo en Etapas

Para llegar al resultado final de esta tarea usted avanzará en pasos "[Iterativos y crecientes \(o incrementales\)](#)" comunes en metodologías de desarrollo de software actuales. Usted y su equipo irán desarrollando etapas donde los requerimientos del sistema final son abordados gradualmente. En cada etapa usted y su equipo obtendrá una solución que funciona para un subconjunto de requerimientos finales o bien es un avance hacia ellos. En AULA se dispondrá el recurso para subir la solución correspondiente a cada etapa del desarrollo. **Su equipo deberá generar una solución para cada una de las etapas** aun cuando la última integre las primeras. **Los archivos readme.txt y de documentación deben ser preparados sólo para la última etapa. Prepare en su repositorio Git un proyecto IntelliJ o Visual Studio Code para cada una de las etapas.** Esto tiene por objetivo conocer la metodología de desarrollo iterativo e incremental.

4.1. Etapa 1: Lectura de configuración y despliegue de sólo celulares

Desarrolle las clases Stage1, Cellular, CellularView, Equipo, Territory y TerritoryView necesarios para leer el archivo de configuración, generar la ventana como en Figura 1 e incluir la barra de menú aunque no se responda a ninguna de sus opciones.

Para comenzar usted puede revisar el código de ayuda [aquí disponible](#) y su [diagrama de clases](#).

4.2. Etapa 2: Despliegue de todos los equipos y activación del menú de simulación

Cree la clase Stage2 y agregue las clases EloTelTag, EloTelTagView, Tablet, TabletView, y otras que estime para mostrar todos los equipos en el territorio y para que vía menú Simulation sea posible iniciar y pausar la simulación. En esta etapa los equipos no reportan sus posiciones.

Para alguno de los requerimientos de esta etapa usted puede revisar el código de [ayuda aquí disponible](#) y su [diagrama de clases](#).

4.3. Etapa 3: Reporte de posiciones hacia ETnube

Cree la clase Stage3 y agregue la clase ETNube. En esta etapa los celulares deben reportar a la nube las posiciones de los equipos rastreables cercanos y un menú emergente debe mostrar la

posición de los equipos según la opción FindMy. Tags y Tablets muestran su señal de radar, pero no emiten sonido. La ventana con la información de los equipos de esa persona no se actualiza.

4.4. Etapa 4: **Cumplimiento de todas las funcionalidades**

Complete lo necesario para cumplir con las funcionalidades descritas en el punto 3. Usando notación compatible con el utilitario javadoc, en esta etapa su equipo debe documentar las clases EloTelTag, EloTelTagView y sus superclases hasta Equipo si corresponde.

4.5. **Extra-crédito:** Si desarrolla esta parte, lo debe indicar en su readme. Su equipo puede recibir hasta 8 puntos adicionales si el menú emergente incluye la opción GFindMy, por Graphics Find My. Al seleccionar esta opción se muestra una ventana con la misma imagen de la ventana principal de la simulación y vistas de los equipos de la persona. Las vistas de los equipos se deben actualizar cada un segundo para reflejar la última información disponible en la nube.

5. **Elementos por considerar en su entrega**

En AULA habrá un buzón para subir únicamente el enlace al repositorio de GitHub. En dicho repositorio deberá incluir el readme y la documentación, indicando todas las etapas desarrolladas y la información necesaria para revisar y ejecutar el proyecto. **Su equipo deberá entregar una solución para cada una de las etapas** aun cuando la última integre las primeras. **El readme y archivo de documentación deben entregarse sólo para la última etapa desarrollada. En su readme indique cómo generar la documentación con javadoc.**

En su archivo de documentación (pdf o html) incorpore el diagrama de clases de su aplicación (última etapa)

Revise lo indicado en "[Normas de Entrega de Tareas](#)", en caso de diferencias, lo señalado en este escrito tiene precedencia.

6. **Sobre la arquitectura Modelo Vista Controlador**

Para organizar interfaces gráficas una "solución de software general recomendada" (éstas son conocidas como [patrones de diseño](#)) es el patrón "[modelo-vista-controlador](#)".

Un **modelo** es una clase que caracteriza a un objeto y almacena los datos significativos de éste. Por ejemplo, en este caso la clase Celular contiene el modelo de un celular. Por otro lado, tenemos las **vistas**, estas clases indican cómo un objeto se muestra visualmente. En esta tarea las vistas son construidas con elementos gráficos de JavaFX. Se podría tener varias vistas para un mismo modelo, para el caso de un celular usamos sólo una vista CellularView.

Finalmente tenemos el **controlador** del objeto gráfico. Las clases controladoras son aquellas que modifican los datos, por ejemplo, a través de las acciones del usuario en la interfaz. Generalmente las clases controladoras corresponden a los "handlers" o "listeners", los cuales pueden ser implementados vía expresiones lambda. Para un celular podemos considerar a Territory como una clase controladora pues cambia la posición del celular en uno de sus métodos.